

From 1D Self-Similar Diffusion to 2D Cylindrical Self-Similar Diffusion

1 Starting point: 2D cylindrical diffusion (dimensional form)

We begin from the *dimensional* 2D (cylindrical) matter–radiation diffusion system (your file “2D Diffusion - Linear Case”):

$$\frac{\partial E(r, z, t)}{\partial t} - \frac{c}{3\sigma} \left(\frac{\partial^2 E}{\partial r^2} + \frac{1}{r} \frac{\partial E}{\partial r} + \frac{\partial^2 E}{\partial z^2} \right) = c\sigma (U - E), \quad (1)$$

$$\frac{1}{\varepsilon} \frac{\partial U(r, z, t)}{\partial t} = c\sigma (E - U). \quad (2)$$

This corresponds to Eqs. (26)–(27) in the 2D PDF.

2 Insert the 1D self-similar physics (opacity + EOS) into 2D

From the 1D self-similar file, we assume the standard power-law relations:

$$\frac{1}{\kappa_R} = g T^\alpha \rho^{-\lambda}, \quad (3)$$

$$e = f T^\gamma \rho^{-\mu}, \quad U_m = \rho e = f T^\gamma \rho^{-\mu+1}. \quad (4)$$

Also $U_R(T) = aT^4$ and we define

$$\beta(T) \equiv \frac{\partial U_R}{\partial U_m}. \quad (5)$$

As in the 1D derivation, the diffusion coefficient is

$$D(T) \equiv \frac{c}{3\sigma(T)} = \frac{c}{3} g T^\alpha \rho^{-\lambda-1}. \quad (6)$$

And (same algebra as in the 1D file) one gets

$$\beta(T) = \frac{4a\rho^{\mu-1}}{f\gamma} T^{4-\gamma}. \quad (7)$$

What changes from (linear) 2D to (self-similar) 2D? In the 2D linear PDF, σ is treated as constant in the dimensional scheme, producing constant λ_r, λ_z . Here we keep the *same cylindrical operator and discretization*, but we replace the constant diffusion coefficient $\frac{c}{3\sigma}$ by $D(T)$ and replace the “ ε ”-style coupling by the self-similar $\beta(T)$ update (exactly like the 1D self-similar implicit coupling step).

3 Time-lagged coefficients (use the “units” logic)

Following your 1D self-similar implementation philosophy: at time level n , U_R^n is known, hence T^n is known, so D^n and β^n are known coefficients for the implicit step.

From the 1D file:

$$T_{i,j}^n = \left(\frac{U_{R,i,j}^n}{a} \right)^{1/4}, \quad D_{i,j}^n = \frac{c}{3} g (T_{i,j}^n)^\alpha \rho^{-\lambda-1}, \quad \beta_{i,j}^n = \frac{4a\rho^{\mu-1}}{f\gamma} (T_{i,j}^n)^{4-\gamma}. \quad (8)$$

This is the direct 2D analogue of Eqs. (19)–(21) in the 1D self-similar PDF.

4 Discretization on a (z, r) grid (same stencil as your 2D PDF)

Let $z_i = i\Delta z$ and $r_j = j\Delta r$ with $j \geq 0$. Use fully implicit time stepping for E :

$$\frac{E_{i,j}^{n+1} - E_{i,j}^n}{\Delta t} - \nabla \cdot (D^n \nabla E^{n+1}) = \chi c \sigma_{i,j}^n (U_{i,j}^{n+1} - E_{i,j}^{n+1}), \quad (9)$$

where the cylindrical divergence gives the same structure as in the 2D PDF:

$$\nabla \cdot (D \nabla E) = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial E}{\partial r} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial E}{\partial z} \right).$$

We now discretize it using face fluxes (finite-volume style), matching the *spirit* of the 2D discretization (radial Laplacian + $(1/r)\partial_r$) while allowing variable D .

4.1 Key step: “fit Eq.(23) into Eq.(21)” via harmonic face diffusion

In 1D you used the decimal average at cell faces in the 1D self-similar PDF):

$$D_{i+\frac{1}{2}}^n = \frac{D_i^n + D_{i+1}^n}{2}. \quad (10)$$

To extend this consistently to 2D, we define decimal averages on *both* coordinate faces:

$$D_{i+\frac{1}{2},j}^n \equiv \frac{D_{i,j}^n + D_{i+1,j}^n}{2}, \quad (11)$$

$$D_{i,j+\frac{1}{2}}^n \equiv \frac{D_{i,j}^n + D_{i,j+1}^n}{2}. \quad (12)$$

This is *exactly* the 2D generalization of Eq. (10), and it is what you want when D varies strongly with T .

4.2 Define the dimensionful “ λ ” factors (units part from the 2D PDF)

Your 2D dimensional scheme defines (Eqs. (32)–(33) there):

$$\lambda_r = \frac{c\Delta t}{3\sigma \Delta r^2}, \quad \lambda_z = \frac{c\Delta t}{3\sigma \Delta z^2}, \quad \gamma_j = \frac{c\Delta t}{3\sigma 2r_j \Delta r} = \frac{\lambda_r}{2j}.$$

In the self-similar case, σ varies, so the natural replacement is to use the *face* diffusion coefficients:

$$\lambda_{z,i+\frac{1}{2},j}^n \equiv \frac{\Delta t}{\Delta z^2} D_{i+\frac{1}{2},j}^n, \quad (13)$$

The cylindrical $(1/r)\partial_r$ contribution can be handled in the following way:

$$\left[\frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial E}{\partial r} \right) \right]_{ij} \approx \frac{1}{r_j \Delta r} \left(r_{j+\frac{1}{2}} D_{i,j+\frac{1}{2}}^n \frac{E_{i,j+1} - E_{i,j}}{\Delta r} - r_{j-\frac{1}{2}} D_{i,j-\frac{1}{2}}^n \frac{E_{i,j} - E_{i,j-1}}{\Delta r} \right) \quad (14)$$

$$\lambda_{r,i\pm\frac{1}{2},j}^n \equiv \frac{\Delta t}{\Delta r^2} \frac{r_{j\pm\frac{1}{2}}}{r_j} D_{i,j\pm\frac{1}{2}}^n. \quad (15)$$

Notice that the definition of $r_{j+1/2}$ is consistent with the grid (e.g. $r_{j+1/2} = (j+1/2)\Delta r$).

4.3 Implicit coupling update (self-similar replacement of ε -form)

From the 1D self-similar file, the matter equation is written in terms of U_R and β :

$$\frac{1}{\beta_{i,j}^n} \frac{U_{R,i,j}^{n+1} - U_{R,i,j}^n}{\Delta t} = \chi c \sigma_{i,j}^n \left(E_{i,j}^{n+1} - U_{R,i,j}^{n+1} \right).$$

Solve it locally (same algebra as your 1D Eqs. (29)–(30)):

$$A_{i,j}^n \equiv \beta_{i,j}^n \Delta t \chi c \sigma_{i,j}^n, \quad (16)$$

$$U_{R,i,j}^{n+1} = \frac{A_{i,j}^n E_{i,j}^{n+1} + U_{R,i,j}^n}{1 + A_{i,j}^n}. \quad (17)$$

This is the exact self-similar analogue of the $f = \frac{1}{1+c\sigma\varepsilon\Delta t}$ trick in the 2D PDF (but with β^n replacing ε).

5 Final 2D implicit stencil for $E_{i,j}^{n+1}$

Insert (17) into the E equation coupling term (RHS of (9)):

$$\chi c \sigma_{i,j}^n (U_{R,i,j}^{n+1} - E_{i,j}^{n+1}) = \frac{\chi c \sigma_{i,j}^n}{1 + A_{i,j}^n} U_{R,i,j}^n - \frac{\chi c \sigma_{i,j}^n}{1 + A_{i,j}^n} E_{i,j}^{n+1}.$$

Therefore the diagonal gains an extra

$$+ \frac{\chi c \sigma_{i,j}^n}{1 + A_{i,j}^n}$$

exactly like the 1D tridiagonal derivation.

Using face fluxes with face D , the fully implicit 5-point form becomes:

$$\begin{aligned} & \left[\frac{1}{\Delta t} + \frac{\lambda_{z,i+\frac{1}{2},j}^n + \lambda_{z,i-\frac{1}{2},j}^n}{\Delta t} + \frac{\lambda_{r,i,j+\frac{1}{2}}^n + \lambda_{r,i,j-\frac{1}{2}}^n}{\Delta t} + \frac{\chi c \sigma_{i,j}^n}{1 + A_{i,j}^n} \right] E_{i,j}^{n+1} \\ & - \frac{\lambda_{z,i+\frac{1}{2},j}^n}{\Delta t} E_{i+1,j}^{n+1} - \frac{\lambda_{z,i-\frac{1}{2},j}^n}{\Delta t} E_{i-1,j}^{n+1} - \frac{\lambda_{r,i,j+\frac{1}{2}}^n}{\Delta t} E_{i,j+1}^{n+1} - \frac{\lambda_{r,i,j-\frac{1}{2}}^n}{\Delta t} E_{i,j-1}^{n+1} \\ & = \frac{1}{\Delta t} E_{i,j}^n + \frac{\chi c \sigma_{i,j}^n}{1 + A_{i,j}^n} U_{R,i,j}^n \quad (\text{interior cells}). \end{aligned} \quad (18)$$

This is the direct 2D generalization of the 1D self-similar tridiagonal equation, with the *same* decimal-face idea now applied in both r and z .

6 Radial diffusion operator and boundary conditions

We consider the cylindrical diffusion operator appearing in the radiation equation,

$$\mathcal{L}_r[E] = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(r, z, t) \frac{\partial E}{\partial r} \right), \quad (19)$$

defined on a uniform radial grid

$$r_j = j \Delta r, \quad j = 0, \dots, N_r - 1.$$

To ensure conservation and stability for variable diffusion coefficient $D(T)$, we discretize $\mathcal{L}_r$ in conservative (finite-volume) form using face fluxes.

6.1 Interior radial discretization

For interior cells $1 \leq j \leq N_r - 2$, the radial operator is approximated by

$$\mathcal{L}_r[E]_{i,j} \approx \frac{1}{r_j \Delta r} \left[r_{j+\frac{1}{2}} D_{i,j+\frac{1}{2}}^n \frac{E_{i,j+1}^{n+1} - E_{i,j}^{n+1}}{\Delta r} - r_{j-\frac{1}{2}} D_{i,j-\frac{1}{2}}^n \frac{E_{i,j}^{n+1} - E_{i,j-1}^{n+1}}{\Delta r} \right], \quad (20)$$

where

$$r_{j \pm \frac{1}{2}} = (j \pm \frac{1}{2}) \Delta r,$$

and $D_{i,j \pm \frac{1}{2}}^n$ is the face diffusion coefficient (harmonic, arithmetic, or geometric average).

Defining the effective cylindrical diffusion weights

$$\lambda_{r,i,j+\frac{1}{2}}^n = \Delta t \frac{r_{j+\frac{1}{2}}}{r_j} \frac{D_{i,j+\frac{1}{2}}^n}{\Delta r^2}, \quad \lambda_{r,i,j-\frac{1}{2}}^n = \Delta t \frac{r_{j-\frac{1}{2}}}{r_j} \frac{D_{i,j-\frac{1}{2}}^n}{\Delta r^2}, \quad (21)$$

the implicit radial contribution to the linear system takes the tridiagonal form

$$-\lambda_{r,i,j-\frac{1}{2}}^n E_{i,j-1}^{n+1} + \left(\lambda_{r,i,j-\frac{1}{2}}^n + \lambda_{r,i,j+\frac{1}{2}}^n \right) E_{i,j}^{n+1} - \lambda_{r,i,j+\frac{1}{2}}^n E_{i,j+1}^{n+1}. \quad (22)$$

6.2 Axis boundary at $r = 0$

Physical regularity at the cylindrical axis requires

$$\left. \frac{\partial E}{\partial r} \right|_{r=0} = 0. \quad (23)$$

This symmetry condition is enforced by imposing

$$E_{i,0}^{n+1} = E_{i,1}^{n+1}, \quad (24)$$

which removes the singularity of the $1/r$ term and ensures zero radial flux through the axis.

In matrix form, the axis row is written as

$$E_{i,0}^{n+1} - E_{i,1}^{n+1} = 0,$$

yielding a stable and conservative discretization.

6.3 Outer radial boundary at $r = R$

At the outer boundary $r = R$, different physical models correspond to different boundary conditions.

6.3.1 Neumann boundary (no radial flux)

For a reflecting or insulating boundary, we impose

$$\left. \frac{\partial E}{\partial r} \right|_{r=R} = 0, \quad (25)$$

which implies zero radial flux through the outer surface.

In conservative form, the outer face flux vanishes,

$$r_{j+\frac{1}{2}} D_{j+\frac{1}{2}} \frac{E_{j+1} - E_j}{\Delta r} = 0,$$

so only the inner face contributes.

The resulting discrete equation at $j = N_r - 1$ is

$$\lambda_{r,i,j-\frac{1}{2}}^n E_{i,j}^{n+1} - \lambda_{r,i,j-\frac{1}{2}}^n E_{i,j-1}^{n+1}, \quad (26)$$

which corresponds to the matrix row

$$E_{i,j}^{n+1} - E_{i,j-1}^{n+1} = 0 \quad (\text{up to scaling}).$$

6.3.2 Dirichlet boundary (fixed bath)

For a fixed radiation bath at $r = R$, we impose

$$E^{n+1}(R) = E_R. \quad (27)$$

The missing outer neighbor is eliminated by substituting $E_{j+1} = E_R$ into the radial flux, yielding

$$\left(\lambda_{r,i,j-\frac{1}{2}}^n + \lambda_{r,i,j+\frac{1}{2}}^n \right) E_{i,j}^{n+1} - \lambda_{r,i,j-\frac{1}{2}}^n E_{i,j-1}^{n+1} = \lambda_{r,i,j+\frac{1}{2}}^n E_R. \quad (28)$$

This introduces an additional diagonal contribution and a source term on the right-hand side, while preserving the tridiagonal structure.

7 Summary of the “fit”

- Take the 2D cylindrical discretization and units definitions from the 2D PDF (dimensional $\Delta t, \Delta r, \Delta z$ and the cylindrical term).
- Replace constant $c/(3\sigma)$ by the self-similar $D(T)$ from the 1D PDF.
- Compute T^n, D^n, β^n from U_R^n (time-lagged coefficients), exactly like 1D.
- *Most importantly:* use the 1D arithmetic-face diffusion Eq. (10) as a template and define arithmetic-face D in both directions (11)–(12); this is the precise meaning of “fit Eq.(23) to Eq.(21)”.
- Apply the axis boundary condition (??) (your 2D Eq. (23)).

8 Adding a gold coat

8.1 Radial discretization on a nonuniform grid

In cylindrical geometry, the radiative diffusion operator in the radial direction takes the form

$$\mathcal{L}_r[E] = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(r, T) \frac{\partial E}{\partial r} \right), \quad (29)$$

where $E = aT^4$ is the radiation energy density and $D = c/(3\sigma)$ is the diffusion coefficient.

Motivation for a nonuniform grid. In the foam region ($0 \leq r < R_{\text{foam}}$), the characteristic length scales are relatively large and a uniform radial spacing Δr is sufficient. In contrast, the gold layer ($R_{\text{foam}} \leq r \leq R$) is thin and highly opaque, leading to steep temperature gradients near the foam–gold interface. To resolve these gradients efficiently, a geometrically stretched grid is used in the gold region, such that the radial cell width increases with r .

As a result, the radial spacing Δr is not constant across the domain and the diffusion operator must be discretized in a form that remains valid on nonuniform grids.

Conservative finite–volume formulation. We discretize the operator using a finite–volume approach, defining fluxes at cell faces:

$$F_{j+\frac{1}{2}} = -r_{j+\frac{1}{2}} D_{j+\frac{1}{2}} \frac{E_{j+1} - E_j}{\Delta r_{j+\frac{1}{2}}}, \quad (30)$$

where

$$\Delta r_{j+\frac{1}{2}} = r_{j+1} - r_j, \quad r_{j+\frac{1}{2}} = \frac{r_j + r_{j+1}}{2}.$$

The divergence of the flux in cell j is then approximated by

$$\mathcal{L}_r[E]_j \approx \frac{F_{j+\frac{1}{2}} - F_{j-\frac{1}{2}}}{r_j \Delta r_j}, \quad \Delta r_j = r_{j+\frac{1}{2}} - r_{j-\frac{1}{2}}. \quad (31)$$

This formulation guarantees:

- exact conservation of radial energy flux,
- consistency across the foam–gold interface,
- automatic reduction to the standard second–order scheme when the grid is uniform.

Axis and outer boundary treatment. At the cylindrical axis ($r = 0$), symmetry implies $\partial E / \partial r = 0$. This is enforced by setting the inward flux $F_{-1/2} = 0$ and retaining only the outward contribution.

At the outer boundary ($r = R$), either a Neumann condition ($\partial E / \partial r = 0$) or a Dirichlet condition ($E = E_{\text{bath}}$) is imposed by appropriately modifying the outward flux term. but it shouldn't matter a lot here where the gold region is applied.

Implications. Although the foam region uses a uniform grid, the same nonuniform discretization is applied throughout the domain. This avoids artificial discontinuities in the numerical operator at $r = R_{\text{foam}}$ and ensures physically consistent transport across the foam–gold interface.

Goal of the changes

The main runtime cost in the 2D implicit diffusion solver is per-timestep work in two places:

1. **Assembling the sparse matrix** for the implicit step, and
2. **Solving the linear system.**

The refactor focuses on reducing Python-level overhead in matrix assembly (the biggest avoidable cost) while preserving the numerical scheme and boundary-condition structure.

What stayed the same (numerics)

The scheme is still backward Euler in time for the radiation energy density E on a (z, r) grid, with temperature-dependent diffusion:

$$\frac{E^{n+1} - E^n}{\Delta t} - \nabla \cdot (D(T^n) \nabla E^{n+1}) + (\text{coupling}) E^{n+1} = \text{rhs}(E^n, U_R^n), \quad (32)$$

where $D(T) = \frac{c}{3\sigma(T)}$ and the coupling is handled with the same lagged- T linearization you were using.

Boundary conditions are still:

- $z = 0$: either Dirichlet “drive” (default) or Marshak-style equation (optional),
- $z = L_z$: Dirichlet bath,
- $r = 0$: symmetry (Neumann),
- $r = R$: selectable Dirichlet bath or Neumann $\partial E / \partial r = 0$.

Big change #1: CSR sparsity-pattern caching

The issue in the original approach

The original implementation constructed the sparse matrix by inserting coefficients inside nested loops (commonly with a format like LIL). Even though the matrix is sparse, inserting entries row-by-row in Python every timestep is slow because:

- the *pattern* (which entries are nonzero) is identical every timestep, but
- the code was re-building that structure repeatedly.

The new idea

Split “build the matrix” into two parts:

1. Build and cache the **CSR pattern** once (row pointers and column indices).
2. Each timestep, allocate a **data** array and fill only the coefficients.

CSR format stores a sparse matrix as three arrays:

- **indptr**: row-start pointers,
- **indices**: column indices for each stored nonzero,
- **data**: numeric values aligned with **indices**.

How `_ensure_csr_template` works

The helper `_ensure_csr_template(marshak_boundary)`:

- Decides which z -rows are unknowns:
 - If Marshak is used at $z = 0$, then $i = 0$ is included in the unknown vector.
 - If Dirichlet drive is used at $z = 0$, then $i = 0$ is excluded (it is imposed after the solve).

- Defines a flattening map from 2D indices (i, j) to a 1D unknown index:

$$\text{idx}(i, j) = (i - i_0) N_r + j.$$

- First pass: for every row, constructs the sorted list of columns corresponding to the 5-point stencil (or a reduced stencil on the Marshak boundary row).
- Second pass: precomputes lookup arrays `pos_self`, `pos_im`, `pos_ip`, `pos_jm`, `pos_jp`.

Those `pos_*` arrays are the key performance trick: they tell you the exact position in `data` where each neighbor coefficient must be written. Then the timestep assembly can do fast assignments like

```
data[pos_ip[rows]] = -D_iph/dz2
data[pos_self[rows]] = diag
```

without any sparse insertions.

Big change #2: precompute radial geometry weights

The r -direction diffusion in cylindrical coordinates has geometric factors like $\frac{1}{r} \partial_r (r \partial_r E)$. On a possibly nonuniform r grid, the finite-volume style coefficients depend on the grid geometry.

The refactor pulls all *geometry-only* multipliers into a one-time precompute:

- axis coefficient (for $j = 0$ symmetry handling),
- interior “minus-half” and “plus-half” weights,
- outer boundary weights.

Then each timestep only multiplies by the local face diffusion D to form the final coefficients. This reduces repeated floating-point work and also makes the assembly code clearer.

Big change #3: optional iterative linear solve (with guardrails)

The default solver remains a direct sparse solve (`spsolve`) to match the reference behavior.

An optional iterative path (`bicgstab`) is provided for speed on larger grids:

- Warm-start with the previous timestep solution $x_0 \approx E^n$.
- Use a cheap Jacobi preconditioner (inverse diagonal).
- Handle SciPy API differences (`rtol/atol` vs `tol`).
- If the iterative method fails to converge (or returns non-finite values), fall back to `spsolve`.
- Optionally check the achieved relative residual and fall back if it is too large.

These guardrails are important because iterative solvers can be fast but may produce inaccurate solutions when the system is ill-conditioned or when tolerances are not strict enough.

What you should expect to improve

- **Assembly time per timestep** should drop significantly because the CSR pattern and neighbor positions are cached.
- **Total runtime** improves most when the grid is moderately large and many timesteps are taken.
- **Memory** remains sparse (zeros are not stored), but the main win is reduced repeated work.

How to use the new knobs (practical)

- For correctness-first runs: keep `linear_solver="direct"`.
- For speed experiments: set `linear_solver="bicgstab"` and tune `linear_tol` and `linear_maxiter`.
- If results look off with the iterative solver, tighten tolerances or rely on the residual-check fallback.
- For a flatter-in- r front (less lateral heat loss): consider `bc_r_outer="neumann0"` rather than a Dirichlet bath at $r = R$.

Summary

The refactor does *not* change the physics model or the discretization. It mainly changes *how* the linear system is assembled:

- build CSR sparsity structure once,
- fill only values each timestep using precomputed positions,
- optionally use an iterative solve with safe fallback.

9 Energy Calculations

9.1 Axisymmetric Energy Integral

For an axisymmetric cylindrical geometry with material internal energy density $U_m(r, z, t)$, the total energy in a region at time t is computed as:

$$E_{\text{total}}(t) = \int_0^{L_z} \int_0^R U_m(z, r, t) \cdot 2\pi r \, dr \, dz \quad [\text{erg}] \quad (33)$$

where $2\pi r$ is the circumferential weight factor for cylindrical coordinates.

9.1.1 Numerical Integration

Using composite trapezoidal rule (from `numpy.trapezoid`):

1. Integrate in r direction (for each z):

$$E_z(z, t) = \int_0^R U_m(z, r, t) \cdot 2\pi r dr \approx \sum_{j=1}^{N_r} U_m(z, r_j, t) \cdot w_j^{(r)} \quad (34)$$

where $w_j^{(r)} = 2\pi r_j$ and the trapezoidal rule accounts for node spacing.

2. Integrate in z direction:

$$E_{\text{total}}(t) = \int_0^{L_z} E_z(z, t) dz \approx \sum_{i=1}^{N_z} E_z(z_i, t) \cdot w_i^{(z)} \quad (35)$$

where $w_i^{(z)}$ are trapezoidal weights in z .

9.1.2 Foam and Gold Regions

The computational domain is subdivided into two material regions:

- **Foam:** $0 \leq r < R_{\text{foam}}$
- **Gold:** $R_{\text{foam}} \leq r \leq R_{\text{foam}} + \delta R_{\text{gold}}$

The foam and gold energies are computed by applying a radial mask mask_r in the integration:

$$E_{\text{foam}}(t) = \int_0^{L_z} \int_0^{R_{\text{foam}}} U_m(z, r, t) \cdot 2\pi r dr dz \quad (36)$$

$$E_{\text{gold}}(t) = \int_0^{L_z} \int_{R_{\text{foam}}}^{R_{\text{foam}} + \delta R_{\text{gold}}} U_m(z, r, t) \cdot 2\pi r dr dz \quad (37)$$

Unit Conversion: The code returns energy in erg; convert to hectojoules (hJ) by:

$$E[\text{hJ}] = E[\text{erg}] \times 10^{-9} \quad (38)$$

—

10 Front Position at a Single Radial Index

10.1 Problem Statement

Given the material temperature profile $T_m(z, r_j, t)$ along the z -axis at a fixed radial location $r = r_j$, compute the location of the thermal front $z_F(r_j, t)$ where the heat pulse transitions from heated to ambient material.

10.2 Maximum Gradient Method

The front is located at the position of maximum temperature gradient:

$$z_F(r_j, t) = z_i^* \quad \text{where} \quad i^* = \arg \max_i \left| \frac{dT_m}{dz} \right|_{z=z_i} \quad (39)$$

Numerical Implementation:

$$i^* = \arg \max_{i=0}^{N_z-2} |T_m(z_{i+1}, r_j, t) - T_m(z_i, r_j, t)| \quad (40)$$

This is robust and does not require a critical temperature threshold.

10.3 Threshold Method

Alternatively, the front is defined as the first location where temperature falls below a threshold:

$$z_F(r_j, t) = \min\{z_i : T_m(z_i, r_j, t) \leq \alpha \cdot T_{\text{cold}}\} \quad (41)$$

where:

- T_{cold} = ambient temperature (default: 300 K for CGS or 300/ $K_{\text{per HeV}}$ for hev—ns)
- α = threshold factor (default: $\alpha = 5$)

If no cell satisfies the condition, z_F is set to the rightmost z -node.

—

11 Surface Front (2D Front Position)

11.1 Problem Statement

Compute the full 2D front surface $z_F(r, t)$ —the front location as a function of both radial and time coordinates.

$$z_F(r_j, t_n) \quad \text{for all } j = 0, 1, \dots, N_r - 1 \text{ and } n = 0, 1, \dots, N_t - 1 \quad (42)$$

This gives an $(N_t \times N_r)$ array describing how the front surface evolves.

11.2 Maximum Gradient Method (2D)

For each radial location r_j and time step t_n , independently apply the 1D max-gradient method:

$$z_F(r_j, t_n) = z_i^*, \quad \text{where } i^* = \arg \max_i |T_m(z_{i+1}, r_j, t_n) - T_m(z_i, r_j, t_n)| \quad (43)$$

Vectorized Implementation:

$$\mathbf{dT} = |\Delta T_m|_{\text{along } z} \Rightarrow \text{shape } (N_t, N_z - 1, N_r) \quad (44)$$

$$\mathbf{i}^* = \arg \max(\mathbf{dT}, \text{axis} = 1) \Rightarrow \text{shape } (N_t, N_r) \quad (45)$$

$$z_F = z[\mathbf{i}^*] \Rightarrow \text{shape } (N_t, N_r) \quad (46)$$

11.3 Threshold Method (2D)

For each (r_j, t_n) pair:

$$z_F(r_j, t_n) = \min\{z_i : T_m(z_i, r_j, t_n) \leq \alpha \cdot T_{\text{cold}}\} \quad (47)$$

Vectorized Implementation:

$$\mathbf{M} = T_m \leq \alpha T_{\text{cold}} \Rightarrow \text{shape } (N_t, N_z, N_r) \quad (48)$$

$$\mathbf{i}^* = \arg \max(\mathbf{M}, \text{axis} = 1) \Rightarrow \text{shape } (N_t, N_r) \quad [\text{first True index}] \quad (49)$$

$$z_F = z[\mathbf{i}^*] \Rightarrow \text{shape } (N_t, N_r) \quad (50)$$

If all temperatures exceed the threshold in a column, z_F defaults to z_{N_z-1} (the rightmost location).

Method	Purpose
<code>compute_energy_foam(stored_Um)</code>	Total foam energy $E_{\text{foam}}(t)$
<code>compute_energy_gold(stored_Um)</code>	Total gold energy $E_{\text{gold}}(t)$
<code>compute_front_at_r(stored_Tm, r_index)</code>	Front at single radius (1D array)
<code>compute_front_surface(stored_Tm)</code>	Front surface $z_F(r, t)$ (2D array)

Code Structure Reference

Example Usage

```
# Create and run simulation
sim = create_simulation(material="SiO2")
stored_t, stored_Um, stored_Tm, stored_TR = run_simulation(sim)

# Compute quantities
energy_foam = sim.compute_energy_foam(stored_Um) * 1e-9 # erg -> hJ
front_r0 = sim.compute_front_at_r(stored_Tm, r_index=0)
front_surface = sim.compute_front_surface(stored_Tm) # shape (Nt, Nr)
```